

Operating System

Process and Thread Management

Topics to be covered

- What is scheduling
- Objectives of scheduling
- Types of scheduler
- Scheduling criteria
- Scheduling algorithms
 - First Come First Served (FCFS)
 - Shortest Job First (SJF)
 - Shortest Remaining Time Next (SRTN)
 - Round Robin (RR)
 - Priority
 - ✓ Preemptive Priority
 - ✓ Non-Preemptive Priority

What is Process scheduling?

- Process scheduling is the activity of the process manager that **handles suspension of running process** from CPU and **selection of another process** on the basis of a particular strategy.
- The **part of operating system** that **makes the choice** is called scheduler.
- The **algorithm used** by this scheduler is called scheduling algorithm.
- Process scheduling is an essential part of a multiprogramming operating systems.
- Process Scheduling handles the selection of a process for the processor on the basis of a scheduling algorithm and also the removal of a process from the processor.

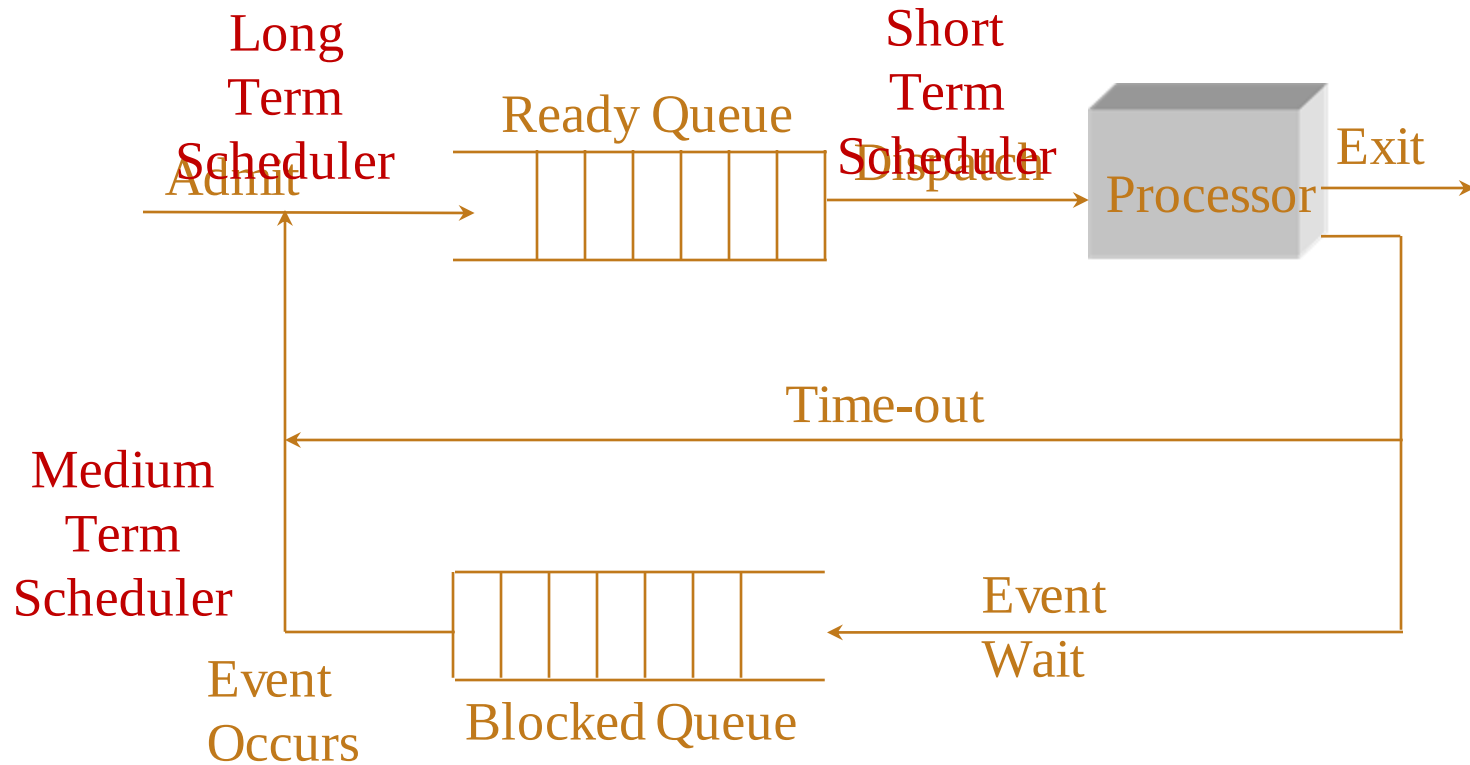
Objectives (goals) of scheduling

- Fairness: giving each process a fair share of the CPU.
- Balance: keeping all the parts of the system busy (Maximize).
- Throughput: no of processes that are completed per time unit (Maximize).
- Turnaround time: time to execute a process from submission to completion (Minimize).
 - $\text{Turnaround time} = \text{Process finish time} - \text{Process arrival time}$

Objectives (goals) of scheduling

- **CPU utilization:** It is **percent of time** that the **CPU is busy** in executing a process.
 - keep CPU as busy as possible (Maximized).
- **Response time:** **time between issuing a command** and **getting the result** (Minimized).
- **Waiting time:** **amount of time a process** has been **waiting** in the ready queue (Minimize).
 - **Waiting time** = Turnaround time – Actual execution time

Types of schedulers



Types of schedulers

Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
It is a job scheduler.	It is a CPU scheduler.	It is a process swapping scheduler.
It selects processes from pool and loads them into memory for execution.	It selects those processes which are ready to execute.	It can re-introduce the process into memory and execution can be continued.
Speed is lesser than short term scheduler.	Speed is fastest among other two schedulers.	Speed is in between both short and long term scheduler.
It is almost absent or minimal in time sharing system.	It is also minimal in time sharing system.	It is a part of time sharing systems.

The job scheduler or long-term scheduler selects processes from the storage pool in the secondary memory and loads them into the ready queue in the main memory for execution.

The short-term scheduler selects one of the processes from the ready queue and schedules them for execution. A scheduling algorithm is used to decide which process will be scheduled for execution next.

The medium-term scheduler swaps out a process temporarily from main memory and places them on secondary memory & vice versa.

Scheduling algorithms

1. First Come First Served (FCFS)
2. Shortest Job First (SJF)
3. Shortest Remaining Time Next (SRTN)
4. Round Robin (RR)
5. Priority
 1. Preemptive
 2. Non-Preemptive

First Come First Served (FCFS)

- Selection criteria
 - The **process** that **request first is served first**.
 - It means that **processes are served in the exact order of their arrival**.
- Decision Mode
 - **Non preemptive**: Once a process is selected, it runs until it is blocked for an I/O or some other event or it is terminated.
- Implementation:
 - This strategy can be **easily implemented** by using FIFO (First In First Out) queue.
 - When CPU becomes free, a process from the first position in a queue is selected to run.

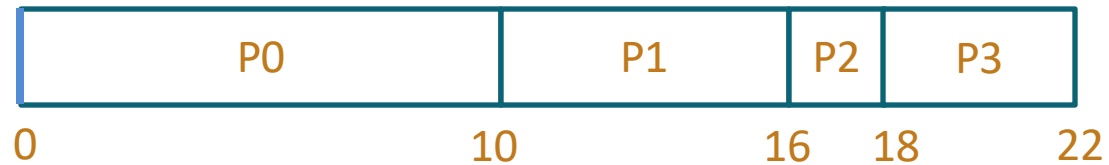


First Come First Served (FCFS)

- Example

Process	Arrival Time (T0)	Time required for completion (ΔT) (CPU Burst Time)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

- Gantt Chart



First Come First Served (FCFS)

Process	Arrival Time (T ₀)	Burst Time (ΔT)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

- Gantt Chart

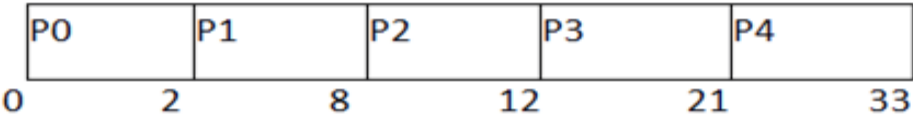


- Average Turnaround Time: $(10+15+15+17)/4 = 14.25$ ms.
- Average Waiting Time: $(0+9+13+13)/4 = 8.75$ ms.

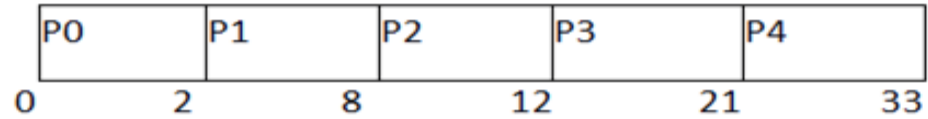
In the Following schedule, there are 5 processes with process ID **P0, P1, P2, P3 and P4**. P0 arrives at time 0, P1 at time 1, P2 at time 2, P3 arrives at time 3 and Process P4 arrives at time 4 in the ready queue. The processes and their respective Arrival and Burst time are given in the following table.

Process ID	Arrival Time	Burst Time
0	0	2
1	1	6
2	2	4
3	3	9
4	4	12

Gantt
Chart



Gantt Chart



Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
0	0	2	2	2	0
1	1	6	8	7	1
2	2	4	12	8	4
3	3	9	21	18	9
4	4	12	33	29	17

Avg Waiting Time=31/5

First Come First Served (FCFS)

■ Advantages

- **Simple** and **fair**.
- **Easy to understand** and **implement**.
- Every process will get a chance to run, so **starvation doesn't occur**.

■ Disadvantages

- **Not efficient** because average waiting time is too high.
- **Convoy effect is possible**. All small I/O bound processes wait for one big CPU bound process to acquire CPU.
- **CPU utilization may be less efficient** especially when a CPU bound process is running with many I/O bound processes.

Shortest Job First (SJF)

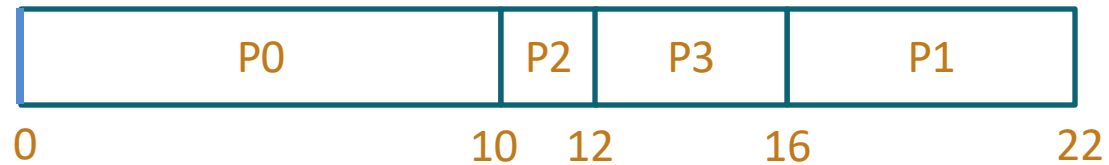
- Selection criteria
 - The process, that **requires shortest time to complete execution**, is **served first**.
- Decision Mode
 - **Non preemptive**: Once a process is selected, it runs until either it is blocked for an I/O or some other event or it is terminated.
- Implementation:
 - This strategy can be **easily implemented** by using FIFO (First In First Out) queue.
 - All processes in a queue are **sorted in ascending order** based on their required CPU bursts.
 - When CPU becomes free, a process from the first position in a queue is selected to run.

Shortest Job First (SJF)

- Example

Process	Arrival Time (T0)	Time required for completion (ΔT) (CPU Burst Time)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

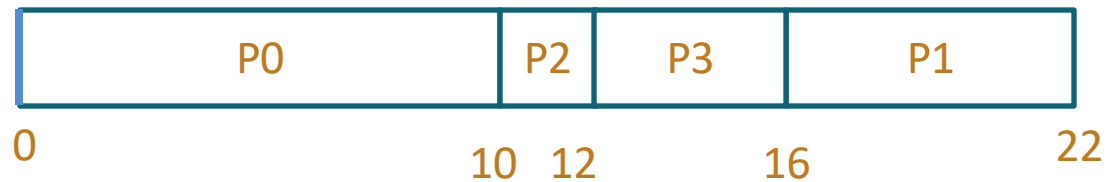
- Gantt Chart



Shortest Job First (SJF)

Process	Arrival Time (T ₀)	Burst Time (ΔT)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

- Gantt Chart



- Average Turnaround Time: $(10+21+9+11)/4 = 12.75$ ms.
- Average Waiting Time: $(0+15+7+7)/4 = 7.25$ ms.

Example: Solve using the SJF

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Solution: we select a process the among available processes with least burst time



Gantt Chart

Process Id	Exit time	Turn Around time	Waiting time
P1	7	$7 - 3 = 4$	$4 - 1 = 3$
P2	16	$16 - 1 = 15$	$15 - 4 = 11$
P3	9	$9 - 4 = 5$	$5 - 2 = 3$
P4	6	$6 - 0 = 6$	$6 - 6 = 0$
P5	12	$12 - 2 = 10$	$10 - 3 = 7$

Now,

- Average Turn Around time = $(4 + 15 + 5 + 6 + 10) / 5 = 40 / 5 = 8$ unit
- Average waiting time = $(3 + 11 + 3 + 0 + 7) / 5 = 24 / 5 = 4.8$ unit



Gantt Chart

Shortest Job First (SJF)

- Advantages:
 - **Less waiting time.**
 - **Good response** for **short processes.**
- Disadvantages :
 - It is **difficult to estimate time required** to complete execution.
 - **Starvation is possible for long process.** Long process may wait forever.

Shortest Remaining Time Next (SRTN)

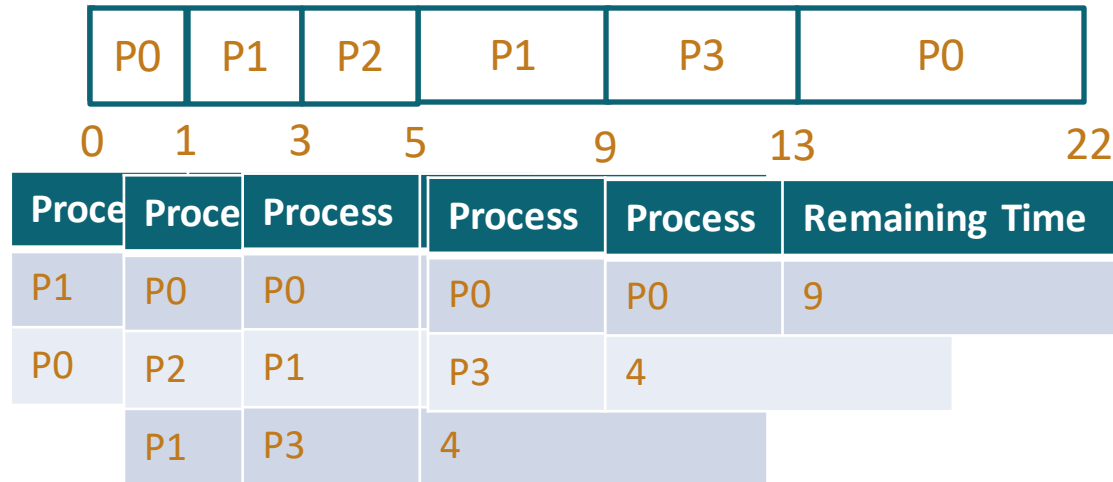
- Selection criteria :
 - The process, **whose remaining run time is shortest**, is **served first**. This is a **preemptive version of SJF scheduling**.
- Decision Mode:
 - **Preemptive**: When a new process arrives, its total time is compared to the current process remaining run time.
 - If the new process needs less time to finish than the current process, the current process is suspended and the new job is started.
- Implementation :
 - This strategy can also be implemented by using sorted FIFO queue.
 - All processes in a queue are **sorted in ascending order on their remaining run time**.
 - When CPU becomes free, a process from the first position in a queue is selected to run.

Shortest Remaining Time Next (SRTN)

Example

Process	Arrival Time (T0)	Time required for completion (ΔT) (CPU Burst Time)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

Gantt Chart



Shortest Remaining Time Next (SRTN)

Process	Arrival Time (T ₀)	Burst Time (ΔT)
P0	0	10
P1	1	6
P2	3	2
P3	5	4



- Average Turnaround Time: $(22+8+2+8) / 4 = 10$ ms.
- Average Waiting Time: $(12+2+0+4)/4 = 4.5$ ms.

Example: Solve using SRTN/SRTF(preemptive SJF)

Process Id	Arrival time	Burst time
P1	0	9
P2	1	4
P3	2	9

Solution: When new process arrived with less Burst time then current CPU is forcefully assigned to this process



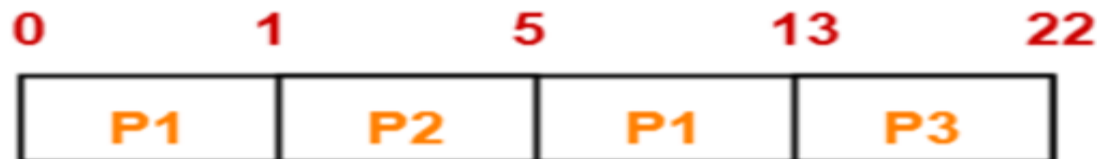
Gantt Chart

Solution: When new process arrived with less Burst time then current CPU is forcefully assigned to this process

Process Id	Exit time	Turn Around time	Waiting time
P1	13	$13 - 0 = 13$	$13 - 9 = 4$
P2	5	$5 - 1 = 4$	$4 - 4 = 0$
P3	22	$22 - 2 = 20$	$20 - 9 = 11$

Now,

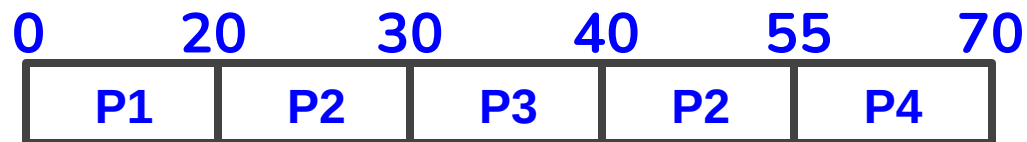
- Average Turn Around time = $(13 + 4 + 20) / 3 = 37 / 3 = 12.33$ unit
- Average waiting time = $(4 + 0 + 11) / 3 = 15 / 3 = 5$ unit



Gantt Chart

Consider the set of 4 processes whose arrival time and burst time are given below- apply CPU scheduling policy is SRTF/SRTN

Process Id	Arrival time	Burst time
P1	0	20
P2	15	25
P3	30	10
P4	45	15

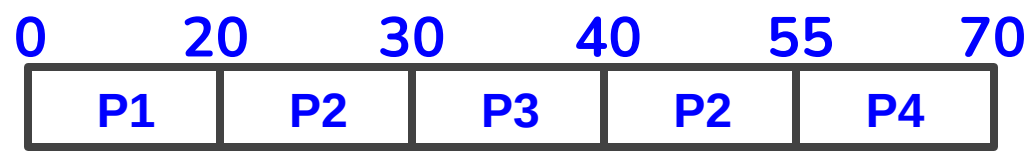


Process Id	Arrival time	Burst time
P1	0	20
P2	15	25
P3	30	10
P4	45	15

CT	TAT	WT
20	20	0
55	40	15
40	10	0
70	25	20

Avg TAT = $20+40+10+25=95/4=23.75$

Avg WT = $0+15+0+20=35/4=8.75$



Shortest Remaining Time Next (SRTN)

- Advantages :
 - **Less waiting time.**
 - **Quite good response** for **short processes.**
- Disadvantages :
 - Again it is **difficult to estimate remaining time** necessary to complete execution.
 - **Starvation is possible** for **long process.** Long process may wait forever.
 - **Context switch overhead** is there.

Round Robin (RR)

- Selection Criteria
 - Each selected process is assigned a time interval, called **time quantum or time slice**.
 - Process is **allowed to run only for this time interval**.
 - Here, two things are possible:
 - First, process is either blocked or terminated before the quantum has elapsed. In this case the CPU switching is done and another process is scheduled to run.
 - Second, process needs CPU burst longer than time quantum. In this case, process is running at the end of the time quantum.
 - Now, it will be preempted and moved to the end of the queue.
 - CPU will be allocated to another process.
 - Here, length of time quantum is critical to determine.

Round Robin (RR)

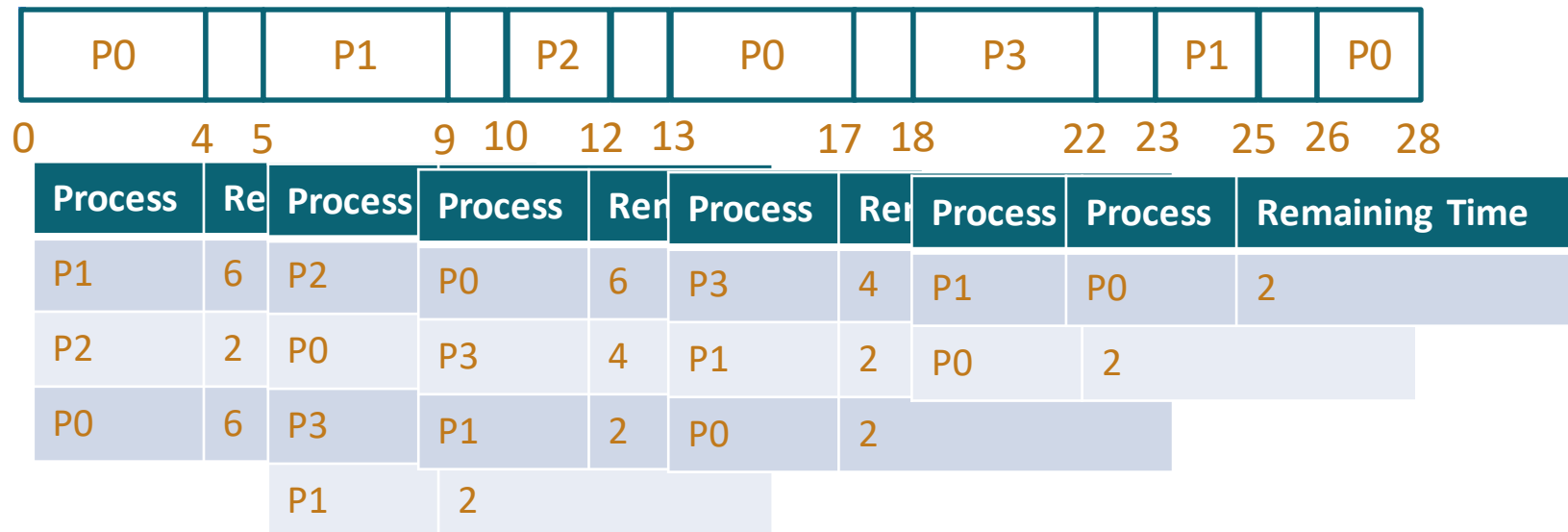
- Decision Mode:
 - **Preemptive**: When quantum time is over or process completes its execution (which ever is earlier), it starts new job.
 - **Selection** of new job is **as per FCFS** scheduling algorithm
- Implementation :
 - This strategy can be implemented by using circular FIFO queue.
 - If any process comes, or process releases CPU, or process is preempted. It is moved to the end of the queue.
 - When CPU becomes free, a process from the first position in a queue is selected to run.

Round Robin (RR)

Example

Process	Arrival Time (T ₀)	Time required for completion (ΔT) (CPU Burst Time)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

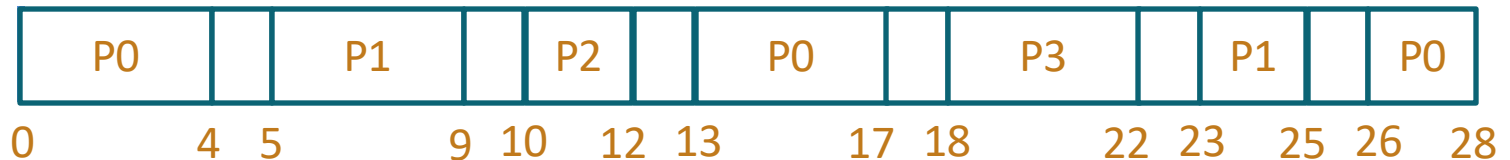
Gantt Chart (Quantum time is 4 ms & context switch overhead is 1 ms)



Round Robin (RR)

Process	Arrival Time (T ₀)	Burst Time (ΔT)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

- Gantt Chart (Quantum time is 4 ms & context switch overhead is 1 ms)

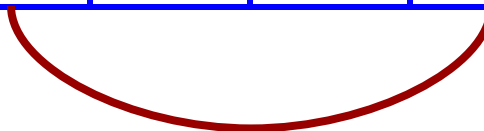
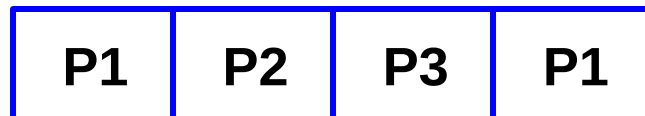


- Average Turnaround Time: $(28+24+9+17)/4 = 19.5$ ms.
- Average Waiting Time: $(18+18+7+13)/4 = 14$ ms.

Another Example: Using RR CPU Scheduling with 2ms Time Quantum

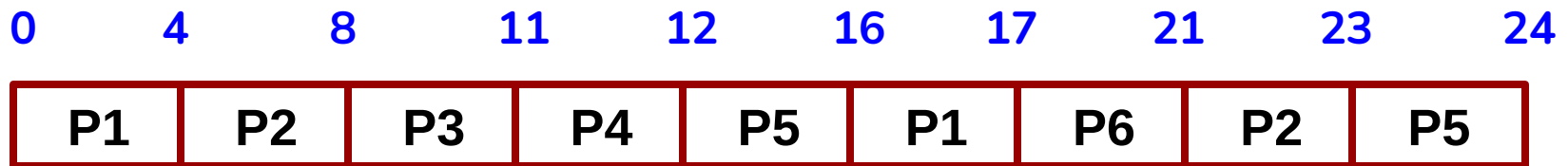
Process Id	Arrival time	Burst time	CT	TAT	WT
P1	0	5	13	13	8
P2	1	3	12	11	8
P3	2	1	5	3	2
P4	3	2	9	6	4
P5	4	3	14	10	7

0 2 4 5 7 9 11 12 13 14



Time Quantum= 4MS

Process	AT	BT	CT	TAT	WT
1	0	5	17	17	12
2	1	6	23	22	16
3	2	3	11	9	6
4	3	1	12	9	8
5	4	5	24	20	15
6	6	4	21	15	11



Round Robin (RR)

- Advantages:
 - **Simplest, fairest** and **most widely used algorithms**.
- Disadvantages:
 - **Context switch overhead** is there.
 - **Determination of time quantum is too critical.**
 - ✓ If it is too **short**, it **causes frequent context switches** and **lowers CPU efficiency**.
 - ✓ If it is too **long**, it **causes poor response** for **short interactive process**.

Non Preemptive Priority Scheduling

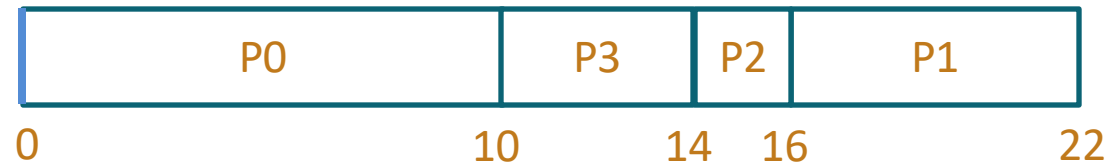
- Selection criteria :
 - The process, that **has highest priority, is served first.**
- Decision Mode:
 - **Non Preemptive**: Once a process is selected, it runs until it blocks for an I/O or some event or it terminates.
- Implementation :
 - This strategy can be implemented by using sorted FIFO queue.
 - All processes in a queue are **sorted based on their priority with highest priority process at front end.**
 - When CPU becomes free, a process from the first position in a queue is selected to run.

Non Preemptive Priority Scheduling

- Example

Process	Arrival Time (T ₀)	Time required for completion (ΔT) (CPU Burst Time)	Priority
P0	0	10	5
P1	1	6	4
P2	3	2	2
P3	5	4	0

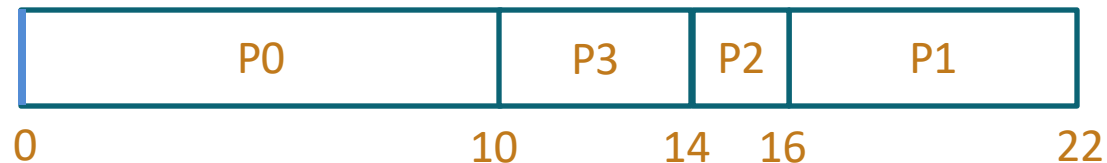
- Gantt Chart (small values for priority means higher priority of a process)



Non Preemptive Priority Scheduling

Process	Arrival Time (T ₀)	Burst Time (ΔT)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

- Gantt Chart (small values for priority means higher priority of a process)



- Average Turnaround Time: $(10+21+13+9) / 4 = 13.25$ ms
- Average Waiting Time: $(0+15+11+5) / 4 = 7.75$ ms

Example: Priority based Non Preemptive

Process	Priority	AT	BT	CT	TAT	WT
1	2	0	3	3	3	0
2	6	2	5	18	16	11
3	3	1	4	7	6	2
4	5	4	2	13	9	7
5	4	6	9	27	21	12
6	7	5	4	11	6	2
7	10	7	10	37	30	20

0 3 7 11 13 18 27 37

P1	P3	P6	P4	P2	P5	P7
----	----	----	----	----	----	----

Non Preemptive Priority Scheduling

- Advantages:
 - Priority is considered so **critical processes can get even better response time.**
- Disadvantages:
 - **Starvation is possible for low priority processes.** It can be overcome by using technique called 'Aging'.
 - **Aging: gradually increases the priority of processes** that wait in the system for a long time.

Preemptive Priority Scheduling

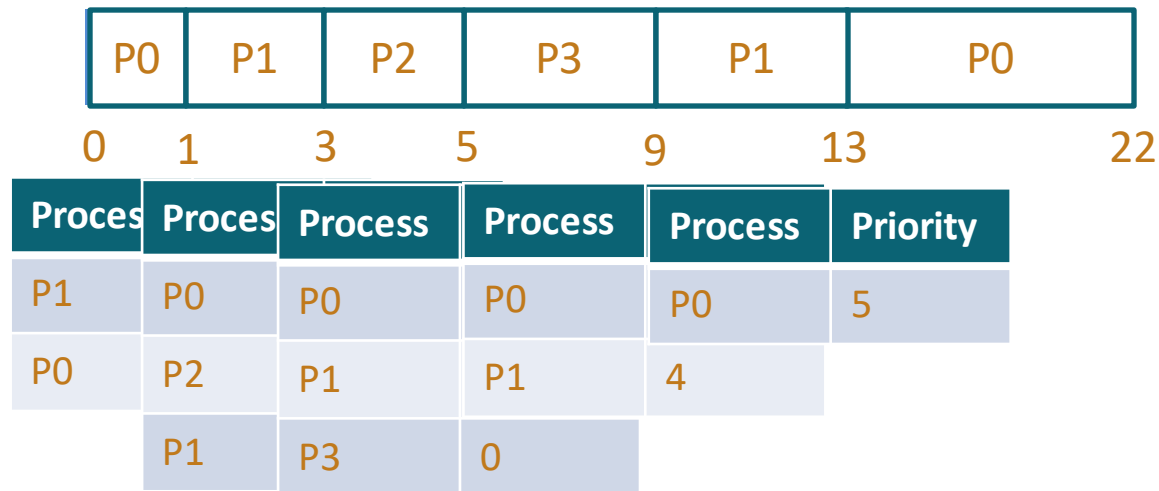
- Selection criteria :
 - The process, that **has highest priority, is served first.**
- Decision Mode:
 - **Preemptive:** When a new process arrives, its priority is compared with current process priority.
 - If the new process has higher priority than the current, the current process is suspended and new job is started.
- Implementation :
 - This strategy can be implemented by using sorted FIFO queue.
 - All processes in a queue are **sorted based on priority with highest priority process at front end.**
 - When CPU becomes free, a process from the first position in a queue is selected to run.

Preemptive Priority Scheduling

- Example

Process	Arrival Time (T ₀)	Time required for completion (ΔT) (CPU Burst Time)	Priority
P0	0	10	5
P1	1	6	4
P2	3	2	2
P3	5	4	0

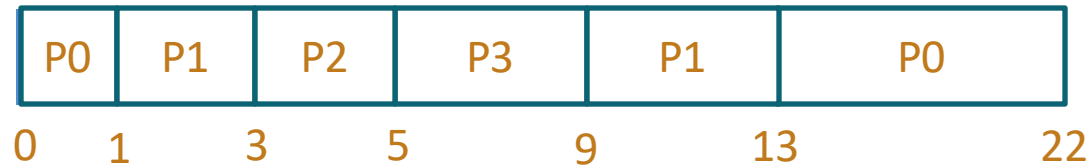
- Gantt Chart (small values for priority means higher priority of a process)



Preemptive Priority Scheduling

Process	Arrival Time (T ₀)	Burst Time (ΔT)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

- Gantt Chart (small values for priority means higher priority of a process)



- Average Turnaround Time: $(22+12+2+4) / 4 = 10$ ms
- Average Waiting Time: $(12+6+0+0) / 4 = 4.5$ ms

Example: Priority based Preemptive

Process	Priority	AT	BT	CT	TAT	WT
1	2	0	1	1	1	0
2	6	1	7	22	21	14
3	3	2	3	5	3	0
4	5	3	6	16	13	7
5	4	4	5	10	6	1
6	10	5	15	45	40	25
7	9	15	8	30	15	7

0 1 2 5 10 16 22 30 45

P1	P2	P3	P5	P4	P2	P7	P6
----	----	----	----	----	----	----	----

Preemptive Priority Scheduling

- Advantages:
 - Priority is considered so **critical processes can get even better response time.**
- Disadvantages:
 - **Starvation is possible for low priority processes.** It can be overcome by using technique called 'Aging'.
 - **Aging: gradually increases the priority of processes** that wait in the system for a long time.
 - **Context switch overhead is there.**

Real Time Operating System

- Real time computing may be defined as that type of computing in which the **correctness of the system depends not only on the logical result of the computation but also on the time at which the results are produced.**
- Real time task may be classified as hard and soft.
- A **hard real time task is one that must meet its deadline**; otherwise **it will cause unacceptable damage** or a fatal error to the system.
- A soft real time task has an associated **deadline that is desirable but not mandatory**; it will not cause unacceptable damage or a fatal error on missing deadline.

Characteristics of Real Time OS

1. **Determinism:** Operations are performed at fixed, predetermined times or within predetermined time intervals.
2. **Responsiveness:** How long, after acknowledgment, the operating system takes to service the interrupt.

It includes the time to begin execution of the interrupt service routine (ISR). If a context switch is necessary, the delay is longer than an ISR executed within the context of the current process.

Characteristics of Real Time OS

3. User Control: User should be able to
 - 1) Specify paging or process swapping
 - 2) Decide which processes must reside in main memory
 - 3) Establish the rights of processes
 - 4) Select algorithms for disks scheduling
4. Reliability: Real time system must be reliable. Reliability means the system should not fail.
5. Fail-soft operation: It is an ability of a system to fail in such a way as to preserve as much capability and data as possible.

Factors of Real Time Scheduling

1. Whether a system performs schedulability analysis or not
2. If it does, whether it is done statically or dynamically
3. Whether the result of the analysis itself produces a schedule or plan according to which task are dispatched at run time.

Classes of Real Time Scheduling Algorithms

1. Static table-driven approaches

- It is applicable to a periodic tasks. This performs a static analysis of feasible schedules.
- Input required for analysis are: periodic arrival times, execution time, periodic ending deadline and relative priority of tasks.
- The scheduler tries to develop schedule to meet all needs.

Classes of Real Time Scheduling Algorithms

2. Static priority-driven preemptive approaches

- A static analysis is done, and the result is used to assign priorities to tasks. A traditional priority driven preemptive scheduler can then be used.
- At run-time, task with highest-priority are executed first, with preemptive-resume policy. When resources are used, need to compute worst-case blocking times.
- Usually, in a real-time system, the priority is related to the time constraints on the tasks.

Classes of Real Time Scheduling Algorithms

3. Dynamic planning-based approaches

- Feasibility is determined at run-time rather than an offline analysis prior to start of execution.
- An arriving task is accepted only if it is feasible to meet its time constraints.
- Requires constant reworking of the schedule to accommodate new tasks and existing ones.

Classes of Real Time Scheduling Algorithms

4. Dynamic Best Effort

- There is no feasibility analysis.
- System tries to meet all deadlines and aborts any started process whose deadline is missed.

Multilevel Queue



- Ready queue is partitioned into separate queues, eg:

foreground (interactive)

background (batch)

- Process permanently in a given queue
- Each queue has its own scheduling algorithm:

foreground -RR

background -FCFS

Multilevel Queue

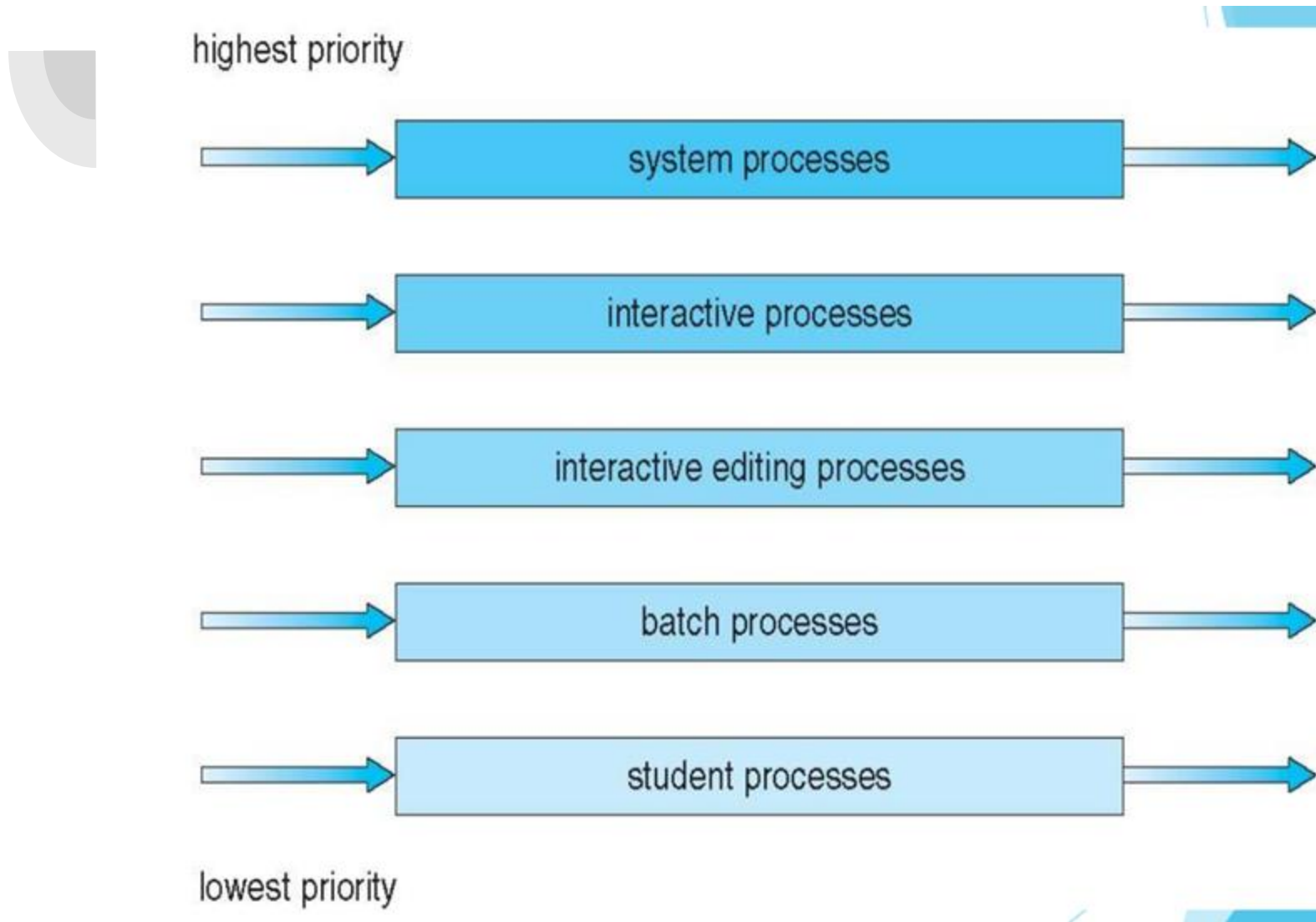
- 
- Scheduling must be done between the queues:

Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.

Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR

20% to background in FCFS

Multilevel Queue Scheduling



Multilevel Feedback Queue



- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:

number of queues

scheduling algorithms for each queue

method used to determine when to upgrade a process

method used to determine when to demote a process

method used to determine which queue a process will enter when that process needs service

Multilevel Feedback Queue

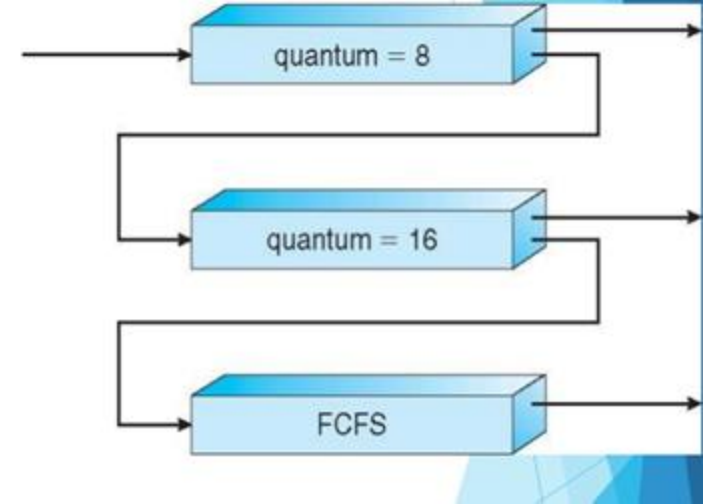
Example:

Three queues:

Q0 – RR with time quantum 8 mill

Q1 – RR time quantum 16 milliseconds

Q2 – FCFS



Scheduling

A new job enters queue Q 0 which is served FCFS When it gains CPU, job receives 8 milliseconds

If it does not finish in 8 milliseconds, job is moved to queue Q 1

At Q 1 job is again served FCFS and receives 16 additional milliseconds

If it still does not complete, it is preempted and moved to queue Q 2

Multiple-Processor Scheduling

- 
- ❖ When multiple processors are available, then the scheduling gets more complicated, because now there is more than one CPU which must be kept busy and in effective use at all times.
 - ❖ **Load sharing** revolves around balancing the load between multiple processors.
 - ❖ Multi-processor systems may be **heterogeneous**, (different kinds of CPUs), or **homogenous**, (all the same kind of CPU).

Approaches to Multiple-Processor Scheduling



asymmetric multiprocessing

in which one processor is the **master**, controlling all activities and running all **kernel code**, while the other runs only **user code**. This approach is relatively simple, as there is no need to share critical system data.

symmetric multiprocessing, SMP

where each processor schedules its own jobs, either from a common ready queue or from separate ready queues for each processor.

all modern OSes support SMP, including XP, Win 2000, Solaris, Linux, and Mac OSX.

Multiple-Processor Scheduling



Processor Affinity

Processors contain cache memory, which speeds up repeated accesses to the same memory locations.

If a process were to switch from one processor to another each time it got a time slice, the data in the cache (for that process) would have to be invalidated and re-loaded from main memory, thereby obviating the benefit of the cache.

Therefore SMP systems attempt to keep processes on the same processor, via *processor affinity*.

Multiple-Processor Scheduling



Soft Affinity

Soft affinity occurs when the system attempts to keep processes on the same processor but makes no guarantees.

hard affinity

in which a process specifies that it is not to be moved between processors.

Linux and some other OSes support *hard affinity*

Multiple-Processor Scheduling

Main memory architecture can also affect process affinity, if particular CPUs have faster access to memory on the same chip or board than to other memory loaded elsewhere. (Non-Uniform Memory Access, NUMA.) As shown below, if a process has an affinity for a particular CPU, then it should preferentially be assigned memory storage in "local" fast access areas.

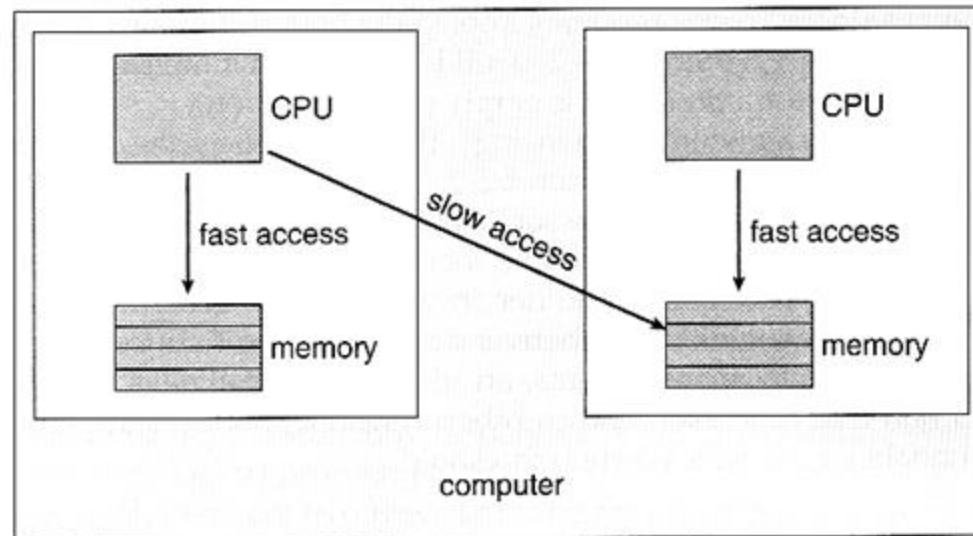


Figure 5.9 NUMA and CPU scheduling.

Load Balancing



Obviously an important goal in a multiprocessor system is to balance the load between processors, so that one processor won't be sitting idle while another is overloaded.

Systems using a common ready queue are naturally self-balancing, and do not need any special handling. Most systems, however, maintain separate ready queues for each processor.

Balancing can be achieved through either ***push migration*** or ***pull migration***:

Push migration involves a separate process that runs periodically, (e.g. every 200 milliseconds), and moves processes from heavily loaded processors onto less loaded ones.

Load Balancing



Balancing can be achieved through either ***push migration*** or ***pull migration***:

Push migration involves a separate process that runs periodically, (e.g. every 200 milliseconds), and moves processes from heavily loaded processors onto less loaded ones.

Pull migration involves idle processors taking processes from the ready queues of other processors.

Push and pull migration are not mutually exclusive.

Exercise-1: Solve Using SJF

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2

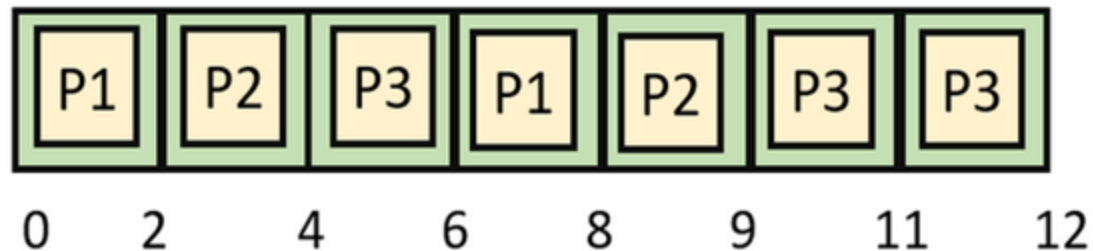
Exercise-1: Priority Scheduling

PROCESS	BURST TIME	PRIORITY
P1	21	2
P2	3	1
P3	6	4
P4	2	3

Exercise-2: Solve Using RR time Quantum is 2ms

Process Queue	Burst Time	Arrival Time
1	4	0
2	3	0
3	5	0

Time Slice = 2



Exercise-3

- Consider the following set of processes with length of CPU burst time given in milliseconds.

○ Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

- Assume arrival order is: P1, P2, P3, P4, P5 all at time 0 and a smaller priority number implies a higher priority. Draw the Gantt charts illustrating the execution of these processes using preemptive priority scheduling.

*Thank
you*

